📌 SQL Subqueries – Complete Notes

🔷 1. What is a Subquery?

A **Subquery** (Inner Query / Nested Query) is a query written inside another SQL query.

- The **inner query executes first**
- Its result is used by the **outer query**

📍 Subqueries can be used in:

- SELECT
- FROM
- WHERE
- HAVING

🔷 2. Types of Subqueries

✅ 1. Single-Row Subquery

- Returns **one row**
- Used with: =, >, <, >=, <=

✅ 2. Multi-Row Subquery

- Returns **multiple rows**
- Used with: IN, ANY, ALL

✅ 3. Scalar Subquery

- Returns **a single value**
- Can be used anywhere a single value is expected

✅ 4. Correlated Subquery

- Refers to columns from the outer query
- Executes **once for each row** of outer query

📦 Dataset: products Table

Columns:

- product_id
- product_name
- brand_name

- category
- price
- discount
- rating
- rating_count

🔷 **3. Basic Query**

📌 **Find Average Rating**

```
SELECT AVG(rating) AS avg_rating
FROM gfg.products;
```

🔷 **4. Subquery in WHERE Clause**

📌 **Brands with Rating Above Average**

```
SELECT brand_name
FROM gfg.products
WHERE rating > (
   SELECT AVG(rating)
   FROM gfg.products
);
```

✓ Inner query finds average rating
✓ Outer query filters brands above that average

🔷 **5. Subquery in SELECT Clause**

📌 **Show Product + Overall Average Rating**

```
SELECT product_name,
      rating,
      (SELECT AVG(rating) FROM gfg.products) AS avg_rating
FROM gfg.products;
```

✓ Scalar subquery
✓ Displays same average value for every row

🔷 6. Subquery with Multiple Results

📌 Total Rating Count Per Brand

```
SELECT brand_name,
      (SELECT SUM(rating_count)
       FROM gfg.products
       WHERE brand_name = p.brand_name) AS total_rating_count
FROM (SELECT DISTINCT brand_name FROM gfg.products) AS p;
```

✓ Outer query gets unique brands
✓ Inner query calculates brand-wise total

🔷 7. Examples of Subquery Types

✅ Example 1: Single-Row Subquery

📌 Product with Highest Price

```
SELECT *
FROM products
WHERE price = (SELECT MAX(price) FROM products);
```

✓ Inner query finds max price
✓ Outer query finds product with that price

✅ Example 2: Multi-Row Subquery

📌 Products from Brands with Price > 1000

```
SELECT *
FROM products
WHERE brand_name IN (
   SELECT DISTINCT brand_name
   FROM products
   WHERE price > 1000
);
```

✓ Subquery returns multiple brand names
✓ Outer query selects all products from those brands

✅ **Example 3: Scalar Subquery**

📌 **Products Above Average Price**

```
SELECT *
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

✓ Average price is single value
✓ Compared with each product

✅ **Example 4: Correlated Subquery**

📌 **Products Above Category Average Price**

```
SELECT product_id, product_name, price, category
FROM products p1
WHERE price > (
  SELECT AVG(price)
  FROM products p2
  WHERE p1.category = p2.category
);
```

✓ Subquery depends on outer query
✓ Runs once per row

✅ **Example 5: Subquery in FROM Clause**

📌 **Average Price per Brand**

```
SELECT brand_name, AVG(price) AS avg_price
FROM (SELECT brand_name, price FROM products) AS sub
GROUP BY brand_name;
```

✓ Inner query acts as temporary table

✅ **Example 6: Subquery in SELECT (Percentage Calculation)**

📌 **Price as % of Most Expensive Product**

```
SELECT product_id,
```

```
    product_name,
    price,
    (price / (SELECT MAX(price) FROM products)) * 100 AS price_percentage
FROM products;
```

✓ Finds maximum price
✓ Calculates percentage for each product

🔥 Important Execution Order

1. Subquery executes first
2. Result is passed to outer query
3. Outer query executes

🎯 Key Operators Used with Subqueries

Operator	Use
=	Single-row comparison
IN	Multiple values
ANY	Compare with any value
ALL	Compare with all values
EXISTS	Checks if subquery returns rows

🧠 Key Differences

Type	Returns	Example
Single-row	One row	MAX(), AVG()
Multi-row	Multiple rows	IN clause
Scalar	One value	SELECT inside SELECT
Correlated	Depends on outer query	Category comparison